

C/C++ SDK 使用说明书

V1.0

目录

1. 简介	6
2. 系统要求	6
3. 安装与卸载	6
4. 快速上手	6
4.1. SDK 开发流程	6
4.2. 简易使用示例	7
5. 接口定义	10
5.1. 回调定义	11
5.2. 函数说明	12
5.2.1. 创建设备实例	12
5.2.2. 销毁设备实例	12
5.2.3. 打开设备	12
5.2.4. 关闭设备	13
5.2.5. 检查设备连接是否打开	13
5.2.6. 检查设备连接是否有效	14
5.2.7. 重启读写器	14
5.2.8. 重置设备	14

5.2.9. 查询设备参数	15
5.2.10. 查询 RFID 参数	15
5.2.11. 查询 UHF 参数	15
5.2.12. 查询 APP 参数	16
5.2.13. 设置设备参数	16
5.2.14. 设置 RFID 参数	17
5.2.15. 设置 UHF 参数	17
5.2.16. 设置 APP 参数	18
5.2.17. 查询 GPIO 模式	18
5.2.18. 设置 GPIO 模式	19
5.2.19. 查询 GPIO 电平状态	19
5.2.20. 设置 GPIO 电平状态	20
5.2.21. 蜂鸣器响	20
5.2.22. 开启异步 RFID 盘点	21
5.2.23. 停止 RFID	21
5.2.24. 读 RFID 标签数据	21
5.2.25. 写 RFID 标签数据	22
5.2.26. 锁 RFID 标签	22

5.2.27. 销毁 RFID 标签.....	23
6. 类型定义.....	23
6.1. 枚举.....	23
6.1.1. device_gpio_index_t.....	23
6.1.2. device_gpio_mode_t.....	24
6.1.3. device_status_t.....	24
6.1.4. antenna_index_t.....	26
6.1.5. antenna_mask_t.....	26
6.1.6. uhf_bank_type_t.....	26
6.1.7. uhf_lock_bank_type_t.....	27
6.1.8. rfid_lock_type_t.....	27
6.2. 结构体.....	27
6.2.1. rfid_inventory_t.....	27
6.2.2. rfid_kill_t.....	29
6.2.3. rfid_lock_t.....	29
6.2.4. rfid_read_t.....	30
6.2.5. rfid_write_t.....	30
6.2.6. device_rfid_tag_t.....	31
附录 1 参数表.....	32
1.1 设备类参数.....	32
1.2 RFID 类参数.....	36
1.3 UHF 类参数.....	40
1.4 APP 类参数.....	42
附录 2 工作区域表.....	45
附录 3 设备状态码.....	46

1. 简介

SDK 是专门为我司设备设计的 API 集合，旨在协助用户高效、快速地进行设备的应用开发。

2. 系统要求

✓ 操作系统：Windows/Linux

3. 安装与卸载

SDK 以库形式提供，无需安装。

4. 快速上手

4.1. SDK 开发流程



4.2. 简易使用示例

```
/**
 * @file example_base_rfid_inventory.c
 * @brief C Example: Basic RFID Inventory
 *
 * Compile:
 * gcc -o example_base_inventory example_base_inventory.c -ldevice_api
 */

#include "device_api.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* RFID Tag Callback */
void on_rfid_tag(device_handle_t dev, const device_rfid_tag_t* tag, void*
userdata) {
    (void)dev;
    (void)userdata;

    char epc_hex[128];
    for (int i = 0; i < tag->epc_len; i++) {
        sprintf(epc_hex + i * 2, "%02X", tag->epc[i]);
    }
    epc_hex[tag->epc_len * 2] = '\0';

    printf("Tag: EPC=%s, RSSI=%.2f dBm, Ant=%d\n",
        epc_hex, tag->rssi * 0.01f, (int)tag->antenna);
}

void on_rfid_stop(device_handle_t dev, uint8_t reason, void* userdata) {
    (void)dev;
```

```

        (void)userdata;
        printf("RFID inventory stopped. Reason: 0x%02X\n", reason);
    }

    /* Disconnect callback */
    void on_disconnect(device_handle_t dev, int will_reconnect, void* userdata) {
        (void)dev;
        (void)userdata;
        printf("Disconnected! Will reconnect: %s\n", will_reconnect ? "yes" : "no");
    }

    /* Reconnect callback */
    void on_reconnected(device_handle_t dev, void* userdata) {
        (void)dev;
        (void)userdata;
        printf("Reconnected!\n");
    }

    /* Log callback */
    void on_log(device_handle_t dev, const char* msg, void* userdata) {
        (void)dev;
        (void)userdata;
        printf("[LOG] %s", msg);
    }

    int main(int argc, char* argv[]) {
        const char* port = (argc > 1) ? argv[1] : "192.168.1.201";

        /* Set callbacks */
        device_callbacks_t cbs;
        memset(&cbs, 0, sizeof(cbs));
        cbs.on_rfid_tag = on_rfid_tag;
        cbs.on_rfid_stop = on_rfid_stop;
        cbs.on_disconnect = on_disconnect;
        cbs.on_reconnected = on_reconnected;
        // cbs.on_log = on_log;
    }

```



```

/* Create device instance */
device_handle_t dev = device_create(&cbs);
if (!dev) {
    fprintf(stderr, "Failed to create device\n");
    return 1;
}

/* Open device */
printf("Opening %s ...\n", port);
device_status_t status = device_open(dev, port, 1);
if (status != DEVICE_OK) {
    fprintf(stderr, "Failed to open device: 0x%02X\n", (int)status);
    device_destroy(dev);
    return 1;
}
printf("Device opened successfully!\n");

/* Set RFID antenna */
printf("Setting RFID antenna...\n");
device_param_value_t antenna;
param_reset(&antenna);
param_append_uint(&antenna, ANTENNA_MASK_1|ANTENNA_MASK_4); // Working
antenna mask
param_append_ushort(&antenna, 0); // Antenna dwell time

status = device_save_rfid_param(dev, RFID_ANTENNA, &antenna);
if (status != DEVICE_OK) {
    fprintf(stderr, "Failed to set rfid antenna: 0x%02X\n", (int)status);
    device_destroy(dev);
    return 1;
}
printf("Antenna set successfully.\n");

/* Start RFID inventory */
printf("Starting RFID inventory...\n");

```

```
status = device_rfid_inventory(dev, NULL);
if (status != DEVICE_OK) {
    fprintf(stderr, "Failed to start inventory: 0x%02X\n", (int)status);
    device_destroy(dev);
    return 1;
}
printf("Inventory started.\n");

/* Wait for user input to stop inventory */
printf("Press Enter to stop...\n");
getchar();

/* Stop inventory */
device_rfid_stop(dev);

/* Close device connection */
device_close(dev);
device_destroy(dev);

printf("Done.\n");
return 0;
}
```

5. 接口定义

设备访问接口涵盖了打开设备、关闭设备、查询/配置参数、GPIO 控制、设备升级、RFID 盘点、RFID 读写标签等操作。

5.1. 回调定义

事件	参数类型	说明
on_log	dev: 设备实例 userdata: 传递回调的用户数据 msg: 日志消息	SDK 日志回调
on_reconnected	dev: 设备实例 userdata: 传递回调的用户数据	设备重连接回调 (TCP)
on_disconnect	dev: 设备实例 userdata: 传递回调的用户数据 will_reconnect: API 是否会重连设	设备断开连接回调
on_rfid_tag	dev: 设备实例 userdata: 传递回调的用户数据 tag: 标签数据	RFID 标签到达数据回调
on_rfid_stop	dev: 设备实例 userdata: 传递回调的用户数据 reason: RFID 停止原因	RFID 停止回调
on_gpio_event	dev: 设备实例 userdata: 传递回调的用户数据 gpio: GPIO 编号, 1 起 state: GPIO 状态	设备 GPIO 状态变化回调
on_ota_progress	dev: 设备实例 userdata: 传递回调的用户数据 percent: OTA 百分比	固件升级进度提示性回调

注： 为提升 SDK 处理效率，请勿在事件处理函数中进行长时间业务处理，
用户应当只做数据接收，并同步给其他线程处理。

5.2. 函数说明

5.2.1. 创建设备实例

	类型	说明
<code>device_handle_t device_create(const device_callbacks_t* cbs)</code>		
参数		
cbs	device_callbacks_t	回调功能集合
返回值		
	device_handle_t	设备句柄
说明		
创建设备实例		

5.2.2. 销毁设备实例

	类型	说明
<code>void device_destroy(device_handle_t dev)</code>		
参数		
dev	device_handle_t	设备句柄
返回值		
无		
说明		
销毁设备实例，释放内存		

5.2.3. 打开设备

	类型	说明
<code>device_status_t device_open(device_handle_t dev, const char* param, uint8_t addr)</code>		
参数		
dev	device_handle_t	设备句柄
param	char*	连接参数： 串口连接：[串口号:波特率] TCP 连接：[IP:端口]

		如： 串口连接：COM1:115200 TCP 连接：192.168.1.201:8090
addr	uint8_t	通讯地址，有效范围 1~250，255 为广播地址
返回值		
	device_status_t	打开设备结果
说明		
打开设备连接，并进行必要的初始化操作。		

5.2.4. 关闭设备

	类型	说明
void device_close(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
无		
说明		
断开设备连接		

5.2.5. 检查设备连接是否打开

	类型	说明
int device_is_connected(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
	int	返回设备连接是否已打开
说明		
仅检查当前连接是否已打开，不进行通信检测		

5.2.6. 检查设备连接是否有效

	类型	说明
device_status_t device_check_connect(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
	device_status_t	返回设备连接检查结果
说明		
通过通信测试检查当前连接是否有效		

5.2.7. 重启读写器

	类型	说明
device_status_t device_restart(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
	device_status_t	返回重启结果
说明		
重启读写器		

5.2.8. 重置设备

	类型	说明
device_status_t device_restore(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
	device_status_t	返回设备重置结果
说明		
设备重置（恢复出厂设置）		

5.2.9. 查询设备参数

	类型	说明
device_status_t device_get_device_param(device_handle_t dev, uint8_t id, device_param_value_t* param_value)		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	设备参数类型
param_val	device_param_value_t*	参数对象，用于存储查询到的参数值。
返回值		
	device_status_t	返回查询结果
说明		
查询设备参数，设备参数定义及值类型见设备参数章节		

5.2.10. 查询 RFID 参数

	类型	说明
device_status_t device_get_rfid_param(device_handle_t dev, uint8_t id, device_param_value_t* param_value)		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	RFID 参数类型
param_val	device_param_value_t*	参数对象，用于存储查询到的参数值。
返回值		
	device_status_t	返回查询结果
说明		
查询 RFID 参数，RFID 参数定义及值类型见 RFID 参数章节		

5.2.11. 查询 UHF 参数

	类型	说明
device_status_t device_get_uhf_param(device_handle_t dev, uint8_t id, device_param_value_t* param_value)		
参数		

dev	device_handle_t	设备句柄
id	uint8_t	UHF 参数类型
param_val	device_param_value_t*	参数对象，用于存储查询到的参数值。
返回值		
	device_status_t	返回查询结果
说明		
查询 UHF 参数，UHF 参数定义及值类型见 UHF 参数章节		

5.2.12. 查询 APP 参数

	类型	说明
device_status_t device_get_app_param(device_handle_t dev, uint8_t id, device_param_value_t* param_value)		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	APP 参数类型
param_val	device_param_value_t*	参数对象，用于存储查询到的参数值。
返回值		
	device_status_t	返回查询结果
说明		
查询 APP 参数，APP 参数定义及值类型见 APP 参数章节		

5.2.13. 设置设备参数

	类型	说明
device_status_t device_set_device_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value, bool save)		
device_status_t device_save_device_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value)		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	设备参数类型
param_val	device_param_value_t*	参数对象，用于存储需要设置的参数值。

saved	bool	是否永久存储（存入设备 Flash）
返回值		
	device_status_t	返回设置结果
说明		
设置设备参数，设备参数定义及值类型见设备参数章节		

5.2.14. 设置 RFID 参数

	类型	说明
<code>device_status_t device_set_rfid_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value, bool save)</code> <code>device_status_t device_save_rfid_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value)</code>		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	RFID 参数类型
param_val	device_param_value_t*	参数对象，用于存储需要设置的参数值。
saved	bool	是否永久存储（存入设备 Flash）
返回值		
	device_status_t	返回设置结果
说明		
设置 RFID 参数，RFID 参数定义及值类型见 RFID 参数章节		

5.2.15. 设置 UHF 参数

	类型	说明
<code>device_status_t device_set_uhf_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value, bool save)</code> <code>device_status_t device_save_uhf_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value)</code>		
参数		
dev	device_handle_t	设备句柄

id	uint8_t	UHF 参数类型
param_val	device_param_value_t*	参数对象，用于存储需要设置的参数值。
saved	bool	是否永久存储（存入设备 Flash）
返回值		
	device_status_t	返回设置结果
说明		
设置 UHF 参数，UHF 参数定义及值类型见 UHF 参数章节		

5.2.16. 设置 APP 参数

	类型	说明
<pre>device_status_t device_set_app_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value, bool save) device_status_t device_save_app_param(device_handle_t dev, uint8_t id, const device_param_value_t* param_value)</pre>		
参数		
dev	device_handle_t	设备句柄
id	uint8_t	APP 参数类型
param_val	device_param_value_t*	参数对象，用于存储需要设置的参数值。
saved	bool	是否永久存储（存入设备 Flash）
返回值		
	device_status_t	返回设置结果
说明		
设置 APP 参数，APP 参数定义及值类型见 APP 参数章节		

5.2.17. 查询 GPIO 模式

	类型	说明
<pre>device_status_t device_get_gpio_mode(device_handle_t dev, device_gpio_mode_t* modes, uint32_t* out_mask)</pre>		
参数		

dev	device_handle_t	设备句柄
modes	device_gpio_mode_t*	用于存储查询的 GPIO 模式结果, 是数据组存储
out_mask	uint32_t*	用于存储查询的 GPIO 的掩码, 1 表示存在 GPIO
返回值		
	device_status_t	返回查询结果
说明		
查询读写器 GPIO 模式		

5.2.18. 设置 GPIO 模式

	类型	说明
<code>device_status_t device_set_gpio_mode(device_handle_t dev, uint32_t mask, const device_gpio_mode_t* modes)</code>		
参数		
dev	device_handle_t	设备句柄
mask	uint32_t*	存储需要配置的 GPIO 的掩码
modes	device_gpio_mode_t*	用于存储需要配置的 GPIO 模式, 是数据组存储
返回值		
	device_status_t	返回查询结果
说明		
设置设备 GPIO 模式		

5.2.19. 查询 GPIO 电平状态

	类型	说明
<code>device_status_t device_get_gpio_state(device_handle_t dev, uint32_t* out_state, uint32_t* out_mask)</code>		
参数		
dev	device_handle_t	设备句柄
out_state	uint32_t*	用于存储查询的 GPIO 状态结果

		掩码
out_mask	uint32_t*	用于存储查询的 GPIO 的掩码, 1 表示存在 GPIO
返回值		
	device_status_t	返回查询结果
说明		
查询设备 GPIO 状态, 以掩码方式输出, 1 为高电平, 0 为低电平		

5.2.20. 设置 GPIO 电平状态

	类型	说明
device_status_t device_set_gpio_state(device_handle_t dev, uint32_t mask, uint32_t state)		
参数		
dev	device_handle_t	设备句柄
mask	uint32_t*	存储需要配置的 GPIO 的掩码
state	uint32_t*	用于存储需要配置的 GPIO 电平掩码
返回值		
	device_status_t	返回查询结果
说明		
设置设备 GPIO 状态, 以掩码方式传递, 1 为高电平, 0 为低电平		

5.2.21. 蜂鸣器响

	类型	说明
device_status_t device_beep(device_handle_t dev, uint32_t duration_ms)		
参数		
dev	device_handle_t	设备句柄
duration_ms	uint32_t	蜂鸣器响时长, 单位 mS
返回值		
	device_status_t	返回设置结果
说明		

蜂鸣器响一次

5.2.22. 开启异步 RFID 盘点

	类型	说明
device_status_t device_rfid_inventory(device_handle_t dev, const rfid_inventory_t* inventory)		
参数		
dev	device_handle_t	设备句柄
inventory	rfid_inventory_t*	盘点参数
返回值		
	device_status_t	返回开启异步盘点结果
说明		
设备进行异步 RFID 盘点，开启成功后，设备盘点到的标签通过 RFID 标签回调传递给应用		

5.2.23. 停止 RFID

	类型	说明
device_status_t device_rfid_stop(device_handle_t dev)		
参数		
dev	device_handle_t	设备句柄
返回值		
	device_status_t	返回停止结果
说明		
用于停止 RFID 操作。常用于异步盘点的手动停止		

5.2.24. 读 RFID 标签数据

	类型	说明
device_status_t device_rfid_read_tag(device_handle_t dev, rfid_read_t* read)		
参数		
dev	device_handle_t	设备句柄

read	rfid_read_t	读 RFID 标签参数, 传递标签读取的参数及读回的数据
返回值		
	device_status_t	返回读 RFID 标签操作结果
说明		
按指定参数进行读 RFID 标签操作, 读取完成后会将读取的数据通过 read 参数返回。 一般使用标签操作接口操作指定标签时, 先调用设置 UHF 参数接口设置 TAG_SELECTn 用于匹配指定标签, 再调用标签操作接口进行操作。		

5.2.25. 写 RFID 标签数据

	类型	说明
<code>device_status_t device_rfid_write_tag(device_handle_t dev, const rfid_write_t* write)</code>		
参数		
dev	device_handle_t	设备句柄
write	rfid_write_t*	写 RFID 标签参数, 传递标签写入的参数及写入的数据
返回值		
	device_status_t	返回写 RFID 标签操作结果
说明		
按指定参数进行写 RFID 标签操作。 一般使用标签操作接口操作指定标签时, 先调用设置 UHF 参数接口设置 TAG_SELECTn 用于匹配指定标签, 再调用标签操作接口进行操作。		

5.2.26. 锁 RFID 标签

	类型	说明
<code>device_status_t device_rfid_lock_tag(device_handle_t dev, const rfid_lock_t* lock)</code>		
参数		
dev	device_handle_t	设备句柄
lock	rfid_lock_t	锁 RFID 标签参数
返回值		

	device_status_t	返回锁标签操作结果
说明		
按指定参数进行锁 RFID 标签操作。 一般使用标签操作接口操作指定标签时，先调用设置 UHF 参数接口设置 TAG_SELECTn 用于匹配指定标签，再调用标签操作接口进行操作。		

5.2.27. 销毁 RFID 标签

	类型	说明
<code>device_status_t device_rfid_kill_tag(device_handle_t dev, const rfid_kill_t* kill)</code>		
参数		
dev	device_handle_t	设备句柄
kill	rfid_kill_t	销毁 RFID 标签参数
返回值		
	device_status_t	返回销毁标签操作结果
说明		
按指定参数进行销毁标签操作。 一般使用标签操作接口操作指定标签时，先调用设置 UHF 参数接口设置 TAG_SELECTn 用于匹配指定标签，再调用标签操作接口进行操作。		

6. 类型定义

6.1. 枚举

6.1.1. device_gpio_index_t

GPIO 编号，用于 GPIO 操作、GPIO 触发等

定义	说明
DEVICE_GPIO1	GPIO1 编号

DEVICE_GPIO2	GPIO2 编号
DEVICE_GPIO3	GPIO3 编号
DEVICE_GPIO3	GPIO3 编号
DEVICE_GPIO4	GPIO4 编号
DEVICE_GPIO5	GPIO5 编号
DEVICE_GPIO6	GPIO6 编号
DEVICE_GPIO7	GPIO7 编号

6.1.2. device_gpio_mode_t

GPIO 模式

定义	说明
DEVICE_GPIO_OUTPUT	GPIO 输出模式
DEVICE_GPIO_INPUT	GPIO 输入模式

6.1.3. device_status_t

设备状态码

定义	说明
DEVICE_OK	成功
DEVICE_ERR_CMD_INVALID_TYPE	无效命令类型
DEVICE_ERR_CMD_INVALID_LEN	无效命令长度
DEVICE_ERR_CMD_INVALID_PARAM	无效命令参数
DEVICE_ERR_CMD_EXEC	命令执行错误
DEVICE_ERR_NO_RESPONSE	命令无响应
DEVICE_ERR_GPIO_OVER_RANGE	GPIO 索引越界
DEVICE_ERR_GPIO_INVALID_MODE	无效 GPIO 模式
DEVICE_ERR_GPIO_OTHER_ERROR	其他 GPIO 错误
DEVICE_ERR_PARAM_INVALID_TYPE	无效参数类型
DEVICE_ERR_PARAM_INVALID_VAL	无效参数值
DEVICE_ERR_PARAM_INVALID_LENGTH	无效参数长度
DEVICE_ERR_PARAM_GET_FAILED	获取参数失败
DEVICE_ERR_PARAM_SET_FAILED	设置参数失败
DEVICE_ERR_PARAM_SAVE_FAILED	保存参数失败
DEVICE_ERR_PARAM_OTHER_ERROR	其他参数错误
DEVICE_ERR_TAG_OTHER_ERROR	标签返回其他错误
DEVICE_ERR_TAG_NOT_SUPPORT	标签返回不支持
DEVICE_ERR_TAG_INSUFFICIENT_PRIVILEGES	标签返回权限不足
DEVICE_ERR_TAG_MEMORY_OVERRUN	标签返回内存溢出
DEVICE_ERR_TAG_MEMORY_LOCKED	标签返回内存已锁
DEVICE_ERR_TAG_CRYPTOGRAPHIC_SUITE_ERROR	标签返回加密套件错误
DEVICE_ERR_TAG_COMMAND_NOT_ENCAPSULATED	标签返回命令未封装
DEVICE_ERR_TAG_RESPONSEBUFFER_OVERFLOW	标签返回响应缓冲区溢出
DEVICE_ERR_TAG_SECURITY_TIMEOUT	标签返回安全超时
DEVICE_ERR_TAG_INSUFFICIENT_POWER	标签返回功率不足
DEVICE_ERR_TAG_LOST	标签丢失
DEVICE_ERR_TAG_NON_SPECIFIC_ERROR	非特定错误
DEVICE_ERR_RFID_INVALID_REGION	无效 RFID 区域
DEVICE_ERR_RFID_INVALID_VALUE	无效 RFID 值
DEVICE_ERR_RFID_EXEC	RFID 执行错误
DEVICE_ERR_RFID_NO_ACK	RFID 无应答
DEVICE_ERR_RFID_INTERNAL_ERROR	RFID 内部错误
DEVICE_ERR_RFID_OTHER_ERROR	其他 RFID 参数
DEVICE_ERR_BUSY	设备忙
DEVICE_ERR_INTERNAL	内部错误

定义	说明
DEVICE_ERR_INVALID_LINK	无效连接

6.1.4. antenna_index_t

天线编号，用于载波测试，标签数据等。

定义	说明
ANTENNA_1	天线 1
ANTENNA_2	天线 2
...	...
ANTENNA_16	天线 16

6.1.5. antenna_mask_t

天线掩码，用于线配置

定义	说明
ANTENNA_MASK_1	天线 1
ANTENNA_MASK_2	天线 2
...	...
ANTENNA_MASK_16	天线 16

6.1.6. uhf_bank_type_t

EPC 标签存储区，用于读写标签等操作

定义	说明
UHF_BANK_RESERVED	6C 标签保留区
UHF_BANK_EPC	6C 标签 EPC 区
UHF_BANK_TID	6C 标签 TID 区
UHF_BANK_USER	6C 标签用户区

6.1.7. uhf_lock_bank_type_t

UHF 标签锁区域，用于锁标签操作

定义	说明
UHF_LOCK_BANK_USER	6C 标签用户区
UHF_LOCK_BANK_TID	6C 标签 TID 区
UHF_LOCK_BANK_EPC	6C 标签 EPC 区
UHF_LOCK_BANK_ACCESSPWD	6C 标签访问密码
UHF_LOCK_BANK_KILLPWD	6C 标签销毁密码

6.1.8. rfid_lock_type_t

UHF 标签锁类型，用于锁标签操作

定义	说明
RFID_LOCK_TYPE_WRITEABLE	可读可写
RFID_LOCK_TYPE_WRITE_ALWAYS	永久可写
RFID_LOCK_TYPE_WRITE_WITH_PASSWD	可密码写
RFID_LOCK_TYPE_NOT_WRITEABLE	永久不可写

6.2. 结构体

6.2.1. rfid_inventory_t

盘点 RFID 标签参数

属性	类型	说明
mask	uint32_t	盘点选项，每个 bit 对应一个功能： bit0: 缓存标签（不传输标签数据） bit1: 请求传输 TID bit2: 请求传输频率 bit3: 请求传输相位

		bit4: 请求传输读取次数 bit5: 请求传输时间戳 bit6: 请求传输 rssi bit8: 启用盘点时长选项 bit9: 启用标签选择选项 bit10: 启用密码选项 bit11: 启用预置密码选项 bit16: 启用温度标签选项 bit17: 启动湿度标签选项 bit18: 启动发光标签选项 bit19: 启用 ImpinJ 标签选项 bit20: 启用 NXP 标签选项链路。 bit24: 启用读标签 bit29: 丢弃 RSSI 小于指定值的标签 bit30: 合并上报
timeout	uint32_t	盘点时长，单位 mS。0 表示一直盘点直到收到 StopRFID 指令。默认值为 0。
select_en	uint32_t	标签选择使能掩码，bit0- select 1, bit1- select 2...
passwd	uint32_t	密码
preset_passwd	uint32_t	预设密码索引
temperature.type	uint32_t	温度标签类型
temperature.args	uint32_t	温度标签参数
humility.type	uint32_t	湿度标签类型
humility.args	uint32_t	湿度标签参数
led.type	uint32_t	温度 LED 类型
led.args	uint32_t	温度 LED 参数
impinj	uint32_t	ImpinJ 标签选项 bit0: 启用 TagFocus bit1: 启用 FastID bit8: 启用 Gen2X
nxp	uint32_t	NXO 标签选项
rsi_threshold	int16_t	RSSI 过滤值，单位 0.01dBm
merge_time	uint32_t	标签合并去重时间，单位 ms
read.bank	uint16_t	标签读取区域

read.wpoint	uint16_t	标签读取字地址
read.wlength	uint16_t	标签读取字长度

6.2.2. rfid_kill_t

销毁 RFID 标签参数

属性	类型	说明
mask	uint32_t	销毁选项，每个 bit 对应一个功能： bit8: 启用销毁超时选项 bit9: 启用标签选择选项 bit10: 启用密码选项 bit11: 启用预置密码选项
timeout	uint32_t	销毁超时，单位 mS。
select_en	uint32_t	标签选择使能掩码，bit0- select 1, bit1- select 2...
passwd	uint32_t	密码
preset_passwd	uint32_t	预设密码索引

6.2.3. rfid_lock_t

锁 RFID 标签参数

属性	类型	说明
mask	uint32_t	锁卡选项，每个 bit 对应一个功能： bit8: 启用锁操作时长选项 bit9: 启用标签选择选项 bit10: 启用密码选项 bit11: 启用预置密码选项
timeout	uint32_t	锁卡超时，单位 mS。
select_en	uint32_t	标签选择使能掩码，bit0- select 1, bit1- select 2...
passwd	uint32_t	密码
preset_passwd	uint32_t	预设密码索引
bank	uint16_t	锁区域
type	uint16_t	锁类型

6.2.4. rfid_read_t

读标签参数

属性	类型	说明
mask	uint32_t	读取选项，每个 bit 对应一个功能： bit8: 启用读取时长选项 bit9: 启用标签选择选项 bit10: 启用密码选项 bit11: 启用预置密码选项
timeout	uint32_t	读取超时，单位 mS。
select_en	uint32_t	标签选择使能掩码，bit0- select 1, bit1- select 2...
passwd	uint32_t	密码
preset_passwd	uint32_t	预设密码索引
bank	uint16_t	读取区域
wpoint	uint16_t	读取字地址
wlength	uint16_t	读取字长度
data	uint8_t[]	读取到的数据

6.2.5. rfid_write_t

写 RFID 标签参数

属性	类型	说明
mask	uint32_t	写卡选项，每个 bit 对应一个功能： bit8: 启用写时长选项 bit9: 启用标签选择选项 bit10: 启用密码选项 bit11: 启用预置密码选项
timeout	uint32_t	写卡超时，单位 mS。
select_en	uint32_t	标签选择使能掩码，bit0- select 1, bit1- select 2...
passwd	uint32_t	密码
preset_passwd	uint32_t	预设密码索引

wblock	uint16_t	块大小
bank	uint16_t	写入区域
wpoint	uint16_t	写入字地址
wlength	uint16_t	写入字长度
data	uint8_t[]	写入的数据

6.2.6. device_rfid_tag_t

RFID 标签数据

属性	类型	说明
type	uint8_t	标签类型
pc	uint16_t	PC 数据
epc	uint8_t[]	EPC 数据
epc_len	uint8_t	EPC 数据长度
tid	uint8_t[]	TID 数据
tid_len	uint8_t	TID 数据长度
data	uint8_t[]	盘点时读取的数据
data_len	uint8_t	盘点时读取的数据长度
antenna	uint8_t	读取标签的天线索引
count	uint32_t	读取次数
rss_i	int16_t	标签 RSSI 值, 单位 0.01dBm
frequency_khz	uint32_t	标签读取频率, 单位 kHz
phase	int16_t	标签读取相位, 单位 0.1°
timestamp_sec	uint32_t	UTC 时间戳, 秒部分
timestamp_ms	uint32_t	UTC 时间戳, 纳秒部分
temperature	int16_t	标签温度值, 单位 0.01°C
humidity	int16_t	标签湿度值, 单位 0.01%

附录 1 参数表

1.1 设备类参数

参数编号	参数名称	读写	值描述
0x00	系统版本信息	RO	byte[4]:版本号
			string: 版本描述
0x01	主板版本信息	RO	byte[4]:版本号
			string: 版本描述
0x02	主板序列号	RO	string: 序列号
0x03	以太网 MAC	RO	byte[6]: MAC 地址
0x04	WIFI MAC	RO	byte[6]: MAC 地址
0x05	BT_MAC	RO	byte[6]: MAC 地址
0x0B	WIFI 固件版本	RO	string: 版本描述
0x10	设备名	RW	string:设备描述
0x11	资产编号	RW	string:资产编号
0x12	设备日期	RW	int: UTC 秒
			int: UTC 纳秒
0x13	设备启动信息	RO	int: 设备上电时间 (秒)
			int: 设备开机原因
0x14	开机事件	RW	bool: 开机事件使能
			array 指令 (完整指令, 支持多条)
0x1E	设备温度	RO	short: 设备温度(0.01 摄氏度)
0x1F	设备 SOC	RO	short: 设备 SOC(0.01%)
0x20	设备地址	RW	byte: 地址, 1~250
0x21	RS232 属性	RW	int: 波特率
			byte: 数据位长度, 只能是 8
			byte: 停止位长度, 可以为 0,1,2
			byte: 校验位, 0 N, 1 O, 2 E
			byte: 保留, 填 0
0x22	RS485 属性	RW	int: 波特率
			byte: 数据位长度, 只能是 8
			byte: 停止位长度, 可以为 0,1,2
			byte: 校验位, 0 N, 1 O, 2 E
			byte: 保留, 填 0

0x24	以太网网络属性	RW	bool: DHCP 使能
			byte[4]: IP 地址
			byte[4]: 子网掩码
			byte[4]: 网关
			byte[4]: DNS
			byte: 工作模式, 0 TCP Server 1 TCP Client 2 UDP Server 3 UDP Client
			short: 本地端口
			string: 远程地址
0x25	WiFi 网络属性	RW	short: 远程端口
			bool: DHCP 使能
			byte[4]: IP 地址
			byte[4]: 子网掩码
			byte[4]: 网关
			byte[4]: DNS
			byte: 工作模式, 0 TCP Server 1 TCP Client 2 UDP Server 3 UDP Client
			short: 本地端口
0x26	WiFi 连接 AP	RW	string: 远程地址
			short: 远程端口
0x27	WiFi 连接信息	RO	string: SSID
			string: 密码
0x27	WiFi 连接信息	RO	string: SSID
			bool: 连接状态

0x2F	默认连接	RW	byte: 默认连接 0: RS232 1: RS485 2: RS232 3: Ethernet 4: WiFi 5: BT
0x51	蜂鸣器参数	RW	bool: 蜂鸣器使能
			short: 蜂鸣器工作时长
			short: 蜂鸣器空闲时长
0x52	通信指示灯参数	RW	bool: 通信指示灯使能
			short: 通信指示灯工作时长
			short: 通信指示灯空闲时长
0x53	标签指示灯参数	RW	bool: 标签指示灯使能
			short: 标签指示灯工作时长
			short: 标签指示灯空闲时长
0x60	GPIO1 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x61	GPIO2 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x62	GPIO3 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）

0x63	GPIO4 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x64	GPIO5 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x65	GPIO6 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x66	GPIO7 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）
0x67	GPIO8 中断触发事件	RW	byte: 边沿类型 0: 不触发 1: 上升沿触发 2: 下降沿触发 3: 任意边沿触发
			array 指令（完整指令，支持多条）

1.2 RFID 类参数

参数编号	参数名称	读写	值描述
0x01	RFID 版本信息	RO	byte[4]:版本号
			string: 版本描述
0x02	RFID SN	RO	string: 序列号
0x0F	RFID 属性	RO	int: 支持的协议掩码 bit0: ISO18000-6B bit1: ISO18000-6C/EPC C1G2 bit2: ISO18000-6D bit4: GB29768 bit5: GJB7377.1 bit16: ISO14443A bit17: ISO14443B bit18: ISO15693
			byte: 天线数量
			short: 最大功率(0.01dBm)
			short: 最小功率(0.01dBm)
			short: 默认功率(0.01dBm)
			short: 功率间隔(0.01dBm)
0x10	RFID 当前协议	RW	int: 配置工作协议掩码，依据设备类型支持一种或多种同时（盘点）工作 bit0: ISO18000-6B bit1: ISO18000-6C/EPC C1G2 bit2: ISO18000-6D bit4: GB29768 bit5: GJB7377.1 bit16: ISO14443A bit17: ISO14443B bit18: ISO15693
0x1E	RFID 温度	RO	short: 设备温度(0.01 摄氏度)
0x20	RFID 空闲模式	RW	bool: 使能
			short: 周期 (ms)
			byte: 空闲比例 0~100

0x50	RFID EAS 事件	RW	byte: 使能 0: 禁用 EAS 1: 启动 EAS 2: 启动 EAS, 并开启开机自动读卡
			short: 数据来源: ISO18K6C: 0: EPC 1: TID
			short: 起始位地址, -1 匹配末尾数据
			array: 掩码
			array: 期望值
			array: 匹配执行指令 (完整指令, 支持多条)
			array: 不匹配执行指令 (完整指令, 支持多条)
0x5F	白名单事件	RW	bool: 使能
			short: 数据来源: ISO18K6C: 0: EPC 1: TID 其他: 0: 标签 UID
			short: 起始位地址
			short: 数据位长度, -1 至末数据
			byte: 格式 0: 原始 1: 转 16 进制字符串输出
			array: 匹配执行指令 (完整指令, 支持多条)
0x60	韦根输出	RW	byte: 使能 0: 禁用韦根输出 1: 启动韦根输出 2: 启动韦根输出, 并开启开机自动读卡
			short: 数据来源: ISO18K6C: 0: EPC 1: TID 其他: 0: 标签 UID
			short: 起始位地址; -1 表示发送末尾数据
			byte: 数据位长度

0x61	特殊格式输出	RW	byte: 使能 0: 禁用特殊格式输出 1: 启动特殊格式输出 2: 启动特殊格式输出, 并开启开机自动读卡
			short: 数据来源: ISO18K6C: 0: EPC 1: TID 其他: 0: 标签 UID
			short: 起始位地址, -1 发送末尾数据
			short: 数据位长度
			byte: 输出格式 0: 原始输出 1: 转 16 进制字符串输出
			byte: 校验方式
			int: 附件数据掩码 bit0: ant bit5: rssi bit6: frequency bit7: phase bit29: SN bit30: AssetID
			array: 帧头
			array: 帧尾
			array: 附加数据分割
0x62	模拟键盘输出	RW	byte: 使能 0: 禁用模拟键盘输出 1: 启动模拟键盘输出 2: 启动模拟键盘输出, 并开启开机自动读卡
			short: 数据来源: ISO18K6C: 0: EPC 1: TID 其他: 0: 标签 UID
			byte: 输出格式 0: 原始输出 1: 转 16 进制字符串输出

			array:帧尾
0x80	RFID 天线状态	RO	int: 天线连接掩码, bit0 天线 1 状态, bit1 天线 2 状态
0x81	天线配置	RW	int 工作天线掩码, bit0 天线 1, bit1 天线 2 short: 天线最长工作时长 (ms)
0x82	所有天线功率配置	WO	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x90	天线 1 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x91	天线 2 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x92	天线 3 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x93	天线 4 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x94	天线 5 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x95	天线 6 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x96	天线 7 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x97	天线 8 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x98	天线 9 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x99	天线 10 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x9A	天线 11 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x9B	天线 12 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x9C	天线 13 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x9D	天线 14 功率	RW	short: 读操作功率(0.01dBm) short: 写操作功率(0.01dBm)
0x9E	天线 15 功率	RW	short: 读操作功率(0.01dBm)

			short: 写操作功率(0.01dBm)
0x9F	天线 16 功率	RW	short: 读操作功率(0.01dBm)
			short: 写操作功率(0.01dBm)
0xF0	预设密钥 1	WO	预设密钥,UHF 4 字节,
0xF1	预设密钥 2	WO	预设密钥
0xF2	预设密钥 3	WO	预设密钥
0xF3	预设密钥 4	WO	预设密钥
0xF4	预设密钥 5	WO	预设密钥
0xF5	预设密钥 6	WO	预设密钥
0xF6	预设密钥 7	WO	预设密钥
0xF7	预设密钥 8	WO	预设密钥
0xF8	预设密钥 9	WO	预设密钥
0xF9	预设密钥 10	WO	预设密钥
0xFA	预设密钥 11	WO	预设密钥
0xFB	预设密钥 12	WO	预设密钥
0xFC	预设密钥 13	WO	预设密钥
0xFD	预设密钥 14	WO	预设密钥
0xFE	预设密钥 15	WO	预设密钥
0xFF	预设密钥 16	WO	预设密钥

1.3 UHF 类参数

参数编号	参数名称	读写	值描述
0x0F	UHF 属性	RO	bitmap[256]: 支持的工作区域 bitmap
			bitmap[256]: 支持的工作模式 bitmap
0x10	UHF 频率参数	RW	byte: region
			byte: 自动跳频模式
			0: 手动跳频
			1: 自动跳频
			2: 随机跳频
			array: 手动跳频表
			short: 跳频间隔(ms)
			int: 起始频率(kHz)
			short: 频率间隔(kHz)

			byte: 信道数量
			int: 起始频率 2(kHz)
			short: 频率间隔 2(kHz)
			byte: 信道数量 2
0x60	UHF EPC 基带参数	RW	byte: 工作模式
			byte: session
			byte: Target
			byte: 初始 Q 值
			byte: 最小 Q 值
			byte: 最大 Q 值
0x61	UHF EPC 算法参数	RW	byte: 算法类型
			byte: 算法参数 1
			short: 算法参数 2
			int: 算法参数 3
0x80	标签选择 规则 1	RW	byte: 与 (0) 或 (1) 关系
			short: 需匹配的存储区 ISO18K6C 0: 保留区 1: EPC 2: TID 3: 用户区
			short: 需匹配的起始位地址
			short: 需要匹配的位长度
			array:需要匹配的数据
0x81	标签选择 规则 2	RW	
0x82	标签选择 规则 3	RW	
0x83	标签选择 规则 4	RW	

0x84	标签选择 规则 5	RW	
0x85	标签选择 规则 6	RW	
0x86	标签选择 规则 7	RW	
0x87	标签选择 规则 8	RW	

1.4 APP 类参数

参数编号	参数名称	读写	值描述
0x01	门控参数	RW	byte: 使能 0: 禁用门禁控制器 1: 启用门禁控制器 2: 启用门禁控制器，并开启开机自动读卡
			int: 锁驱动时长，单位 ms
			int: 刷卡间隔，单位 ms； 两次刷卡的间隔

			int: 门磁延时, 单位 ms, 开门后经过指定时候未关门视为超时
0x02	辅助输出参数	RW	byte: 触发条件 0: 无 1: 辅助继电器输入
			byte: 辅助继电器动作 0: 关闭 1: 打开 2: 常开
			int: 驱动时间, 单位 ms
0x08	门继电器 GPIO	RW	byte: 门继电器 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...
			byte: 类型 0: 常闭 1: 常开
0x09	门传感器 GPIO	RW	byte: 门传感器 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...
			byte: 类型 0: 常闭 1: 常开
0x0A	开门按钮 GPIO	RW	byte: 开门按钮 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...
			byte: 类型 0: 常闭 1: 常开
0x0B	联锁输入 GPIO	RW	byte: 联锁输入 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...

			byte: 类型 0: 常闭 1: 常开
0x0C	辅助输入 GPIO	RW	byte: 辅助输入 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...
			byte: 类型 0: 常闭 1: 常开
0x0D	辅助输出 GPIO	RW	byte: 辅助输出 GPIO 编号 0: 无对应 GPIO 1: GPIO1 2: GPIO2 ...
			byte: 类型 0: 常闭 1: 常开
0x10	MQTT 配 置	RW	byte: 使能 0: 禁用 MQTT 1: 启动 MQTT 2: 启用 MQTT, 并开启开机自动读卡
			string:Broker 地址
			short:Broker 端口
			byte:模式 0: 不加密 1: SSL 单向认证
			string:用户名
			string:密码
			string:上行 topic
			byte: 上行 QOS
			string:下行 topic
			byte: 下行 QOS
0x11	MQTT 心 跳配置		bool: 使能
			int: 心跳间隔
0x12	MQTT 标 签上传配 置		bool: 使能
			int: 标签上报间隔

			int: 标签上报最大数量
			int: 附件数据掩码 bit0: ant bit3: tid bit5: rssi bit6: frequency bit7: phase bit29: SN bit30: AssetID

附录 2 工作区域表

设备支持的工作区域列表由设备出厂决定。

编号	说明	频点数
0	CMIIT 920~925MHz	16
1	FCC 902~928MHz	50
2	ETSI 866~867MHz	4
3	ETSI_HIGH 916~918MHz	3
4	ETSI_HYBRID 866~867MHz 916~918MHz	7
5	KOR 917~921MHz	6
6	JPN_NO_LBT 916~921MHz	4
7	JPN_LBT 916~921MHz	6
8	JPN_NO_LBT_19 916~924MHz	19
9	TWN 920~927MHz	14
10	MYS 919~923MHz	8
11	RUS_LOW 866~867MHz	5
12	RUS_HIGH 916~920MHz	4
13	AUS 920~926MHz	10
14	IND 865~867MHz	4
15	BRA 902~907.5MHz 915~928MHz	35
16	VIE 918~923MHz	6
17	ISR 915~917MHz	1
18	ZAF 916~920MHz	17
19	SGP 920~925MHz	10
20	THA 920~925MHz	10

编号	说明	频点数
240	TEST1 800~1000MHz	197
241	TEST2 840~960MHz	117
242	TEST3 860~960MHz	11
243	TEST4 860~930MHz	8

附录 3 设备状态码

设备状态码

值	说明
0x00	成功
0x01	无效命令类型
0x02	无效命令长度
0x03	无效命令参数
0x04	命令执行错误
0x05	命令无响应
0x11	GPIO 索引越界
0x12	无效 GPIO 模式
0x13	其他 GPIO 错误
0x61	无效参数类型
0x62	无效参数值
0x63	无效参数长度
0x64	获取参数失败
0x65	设置参数失败
0x66	保存参数失败
0x67	其他参数错误
0x80	标签返回其他错误
0x81	标签返回不支持
0x82	标签返回权限不足
0x83	标签返回内存溢出
0x84	标签返回内存已锁
0x85	标签返回加密套件错误
0x86	标签返回命令未封装

0x87	标签返回响应缓冲区溢出
0x88	标签返回安全超时
0x8B	标签返回功率不足
0x8E	标签丢失
0x8F	非特定错误
0x90	无效 RFID 区域
0x91	无效 RFID 值
0x9A	RFID 执行错误
0x9B	RFID 无应答
0x9E	RFID 内部错误
0x9F	其他 RFID 参数
0xFB	设备忙
0xFE	内部错误
0xFF	无效连接